Pertemuan 2

Membangun UI Komponen dengan React & TypeScript

The Modern Foundation (Konsep & Setup)

1. **Pendahuluan: Mengapa Vite & TypeScript?**

   Dalam ekosistem pengembangan web modern, efisiensi dan keamanan kode adalah prioritas utama. Kita beralih dari standar lama (Create React App) ke **Vite** karena beberapa alasan teknis yang krusial bagi industri:

Fitur	CRA (Lama)	Vite (Modern)
Kecepatan Build	Lambat karena memproses seluruh aplikasi	Super Cepat menggunakan *Native ESM*
Konfigurasi	Berat dan kompleks	Sangat ringan dan modular
TypeScript	Harus diatur manual	Mendukung secara bawaan (*Native Support*)

2. **Panduan Setup Project (Step-by-Step)**

   Ikuti langkah-langkah berikut di terminal/command prompt kalian untuk menginisialisasi proyek baru

   1. **Inisialisasi Project:** Jalankan perintah untuk membuat template React dengan TypeScript:

      ```
      npm create vite@latest my-web-app --template react-ts
      ```

   2. **Navigasi Folder:** Masuk ke dalam direktori project yang baru dibuat: **`cd my-web-app`**
   3. **Instalasi Dependencies:** Unduh semua pustaka yang dibutuhkan oleh React: **`npm install`**
   4. **Menjalankan Local Server:** Mulai lingkungan pengembangan: **`npm run dev`** Akses melalui browser di alamat: **`http://localhost:5173`**

3. Memahami Ekstensi File: .jsx vs .tsx

Sebagai pengembang, kalian harus memahami perbedaan mendasar pada ekstensi file yang digunakan dalam project ini:

File	Fungsi Utama
.jsx	Digunakan untuk JavaScript standar yang mengandung sintaks HTML (JSX)
.tsx	Digunakan untuk **TypeScript** yang mengandung sintaks HTML (JSX)

Aturan Penting: Jika di dalam kode TypeScript kalian terdapat elemen HTML (JSX), maka file tersebut **WAJIB**menggunakan ekstensi `.tsx`

4. Bedah Struktur Folder Proyek

Setelah instalasi selesai, folder **`src/`** akan menjadi tempat utama kalian bekerja. Berikut adalah komponen penting di dalamnya:

- **main.tsx (Entry Point):** Gerbang utama aplikasi yang bertugas merender seluruh komponen React ke dalam halaman HTML (index.html).
- **App.tsx (Root Component):** Komponen induk yang menampung seluruh tampilan aplikasi kalian.
- **assets/:** Folder khusus untuk aset statis seperti gambar, ikon, dan logo.
- **components/:** Tempat penyimpanan komponen-komponen kecil yang bersifat *reusable* (dapat digunakan kembali)

Core React Logic (Fundamental)

Setelah berhasil melakukan *setup* project, kita akan mempelajari filosofi utama di balik React. Memahami bagian ini adalah kunci untuk menjadi seorang *Frontend Developer* profesional.

1. Component-Based Architecture (Arsitektur Berbasis Komponen)

Dalam pengembangan web tradisional, kita membangun halaman demi halaman. Di React, kita membangun **Komponen**.

- **Definisi:** Komponen adalah bagian kecil dari UI yang mandiri (terisolasi), dapat digunakan kembali (*reusable*), dan memiliki logika sendiri.
- **Analogi Lego:** Jika sebuah aplikasi adalah istana Lego, maka komponen adalah balok-baloknya. Kita bisa menggunakan balok "jendela" yang sama di sisi kiri maupun kanan istana.
- **Hierarki:** Komponen besar (*Parent*) bisa menampung komponen-komponen kecil (*Children*). Contohnya, komponen Navbar dan Sidebar disusun di dalam komponen Dashboard.

2. Props (Properties): Mengirim Data ke Komponen

Jika Komponen adalah sebuah fungsi, maka **Props** adalah argumen atau input yang dimasukkan ke dalam fungsi tersebut.

- **Tujuan:** Cara mengirim data dari komponen induk (*Parent*) ke komponen anak (*Child*).
- **Sifat Utama (Read-Only):** Props bersifat *Immutable*. Komponen anak hanya bisa membaca data tersebut, tidak boleh mengubahnya.
- **Aliran Data:** Selalu satu arah (*Top-Down*), dari atas ke bawah.

Contoh Implementasi TypeScript: Kita akan membuat komponen **Greeting** yang menerima properti **name**.

```
// components/Greeting.tsx
interface GreetingProps {
  name: string; // Menentukan bahwa 'name' harus berupa teks
}

const Greeting: React.FC<GreetingProps> = ({ name }) => {
  return <h1>Hello, {name}!</h1>;
};

export default Greeting;
```

3. State: "Ingatan" Internal Komponen

Jika Props adalah data yang "dititipkan" dari luar, maka **State** adalah data internal yang dimiliki dan dikelola oleh komponen itu sendiri.

- **Definisi:** Objek JavaScript untuk menyimpan data yang dapat berubah seiring interaksi pengguna, seperti klik tombol atau isi *form*.
- **Sifat Reactive:** Ketika State berubah, React akan secara otomatis menggambar ulang (*re-render*) komponen tersebut agar tampilan selalu sinkron dengan data terbaru.
- **Analogi Saklar:** Komponen lampu memiliki *state* isOn. Jika true, lampu menyala. Hanya lampu itu sendiri yang bisa menekan saklarnya.

Contoh Implementasi useState: Mari kita buat tombol penghitung (*Counter*) sederhana.

```
import { useState } from 'react';

const Counter = () => {
  // count: data saat ini, setCount: fungsi untuk mengubah data
  const [count, setCount] = useState<number>(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Kamu telah mengklik sebanyak {count} kali
    </button>
  );
};
```

4. Perbedaan Cepat: Props vs State

Fitur	Props	State
Sumber	Dikirim oleh Parent	Dikelola sendiri oleh komponen
Perubahan	Tidak bisa diubah (Read-Only)	Bisa diubah menggunakan `set` function
Kegunaan	Konfigurasi komponen	Interaktivitas pengguna

5. Latihan Mandiri: "The Challenge"

- Buatlah sebuah komponen bernama UserCard.tsx.
- Gunakan **Props** untuk menerima data username dan role.
- Tambahkan sebuah **State** berupa tombol "Follow" yang jika diklik akan mengubah teks tombol menjadi "Following".

Industrial Styling (Implementation)

Setelah memahami logika komponen, langkah selanjutnya adalah memberikan "wajah" pada aplikasi kita. Di industri, kita tidak lagi menulis CSS manual yang panjang, melainkan menggunakan *Utility-First CSS*.

1. Mengapa Tailwind CSS v4?

Tailwind CSS v4 adalah versi terbaru yang jauh lebih cepat dan terintegrasi langsung dengan mesin *build* Vite.

- **Zero Configuration:** Tidak memerlukan file **`tailwind.config.js`** yang rumit secara bawaan.
- **Vite Plugin:** Menggunakan **`@tailwindcss/vite`** untuk performa maksimal saat pengembangan.
- **Type-Safe CSS:** Bekerja sangat baik dalam lingkungan TypeScript.

2. Langkah Instalasi & Setup

Jalankan perintah berikut di terminal proyek kalian:

1. **Install Library:**

```
npm install tailwindcss @tailwindcss/vite lucide-react
```

2. **Konfigurasi Vite:** Buka file **`vite.config.ts`** dan tambahkan plugin Tailwind:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite' // Import ini

export default defineConfig({
  plugins: [
    react(),
    tailwindcss(), // Aktifkan plugin di sini
  ],
})
```

3. Inisialisasi CSS

   Buka file **`src/index.css`**, hapus seluruh isi file tersebut agar bersih dari gaya bawaan, lalu masukkan satu baris perintah berikut untuk mengaktifkan Tailwind v4:

```
@import "tailwindcss";
```

4. Memahami Utility-First Workflow

   Di industri, kita sudah jarang membuat file CSS terpisah dengan *class* seperti .btn-primary. Sebagai gantinya, kita menggunakan pendekatan **Utility-First**, di mana kita langsung menuliskan fungsinya di dalam properti className pada komponen React.

   Contoh Implementasi pada App.tsx:

```
function App() {
  return (
    <div className="min-h-screen flex items-center justify-center
bg-slate-100">
      <h1 className="text-3xl font-bold text-blue-600">
        Tailwind v4 Berhasil Terpasang!
      </h1>
    </div>
  );
}
```

Arsitektur Design System & Atomic Design

1. Mengapa Design System?

Dalam skala industri, kita tidak hanya membangun halaman, tetapi membangun sebuah **Sistem**. *Design System* adalah kumpulan komponen standar yang dapat digunakan kembali untuk menjaga konsistensi visual dan mempercepat proses pengembangan aplikasi

Isinya biasanya:

- Warna (color system)
- Typography
- Spacing
- Komponen (button, card, form)
- Interaction (hover, active, dll)

2. Metodologi: Atomic Design

Kita membagi antarmuka menjadi lima hierarki berdasarkan kompleksitasnya:

Hierarki	Definisi	Contoh (Kasus Invofest)
Atoms	Unit terkecil yang tidak bisa dipecah lagi.	Button, NavLink, Heading.
Molecules	Gabungan beberapa atom yang memiliki fungsi tertentu.	CardItem, Collapse (FAQ).
Organisms	Gabungan molekul dan atom yang membentuk bagian besar UI.	Navbar, Header, Footer.
Templates	Struktur tata letak halaman (Layout).	App.tsx sebagai Manager Layout.
Pages	Hasil akhir templat yang sudah diisi data nyata.	Halaman Beranda.

3. **Fase 1: Design Tokens (Standarisasi Variabel)**

   Sebelum membuat komponen, kita harus menentukan **Design Tokens** (nilai dasar seperti warna dan tipografi) di dalam index.css menggunakan fitur @theme dari Tailwind v4.

   Implementasi pada src/index.css:

```
@import "tailwindcss";

@theme {
  /* Brand Colors */
  --color-brand-primary: #7f1d1d;    /* Red-900 */
  --color-brand-secondary: #fca5a5;  /* Red-300 */

  /* Semantic Colors */
  --color-success: #22c55e;
  --color-danger: #ef4444;
}
```

4. **Fase 2: Membangun Komponen Dasar (Atoms)**

   Komponen Atom bersifat **Read-Only** (menerima data lewat Props) dan tidak mengelola logika bisnis yang kompleks.

   Contoh: Komponen Button

```
interface ButtonProps {
  label: string;
  variant?: "primary" | "outline";
  onClick?: () => void;
}

export const Button: React.FC<ButtonProps> = ({ label, variant = "primary" }) => {
  const baseStyle = "px-10 py-3 rounded font-medium transition-all";
  const variants = {
    primary: "bg-brand-primary text-white hover:bg-red-800",
    outline: "border border-brand-primary text-brand-primary hover:bg-red-50"
  };

  return <button className={`${baseStyle} ${variants[variant]}`}>{label}</button>;
};
```

5. **Fase 3: Komponen Interaktif (Molecules)**

   Molekul seringkali membutuhkan **State** untuk mengelola interaktivitas internal, seperti "buka-tutup" pada FAQ.

```
export const Collapse: React.FC<CollapseProps> = ({ title, description }) => {
  // State: "Ingatan" internal untuk status buka/tutup [cite: 49, 185]
  const [isOpen, setIsOpen] = useState(false);

  return (
    <div className="border border-gray-200 rounded-lg">
      <button onClick={() => setIsOpen(!isOpen)} className="w-full px-4 py-3
flex justify-between">
        <span>{title}</span>
        <ChevronDown className={`transition-transform ${isOpen ? "rotate-180" :
""}`} />
      </button>
      {isOpen && <div className="px-4 py-3 bg-gray-50">{description}</div>}
    </div>
  );
};
```

6. **Fase 4: Perakitan Akhir (Organisms & Pages)**

   Pada tahap ini, kita merakit semua komponen di dalam App.tsx. Data dikirim dari atas (Parent) ke bawah (Child) melalui **Props**.

   Struktur Kerja Industri:

   - **Define Tokens**: Tentukan warna dan font di CSS.
   - **Build Atoms**: Buat tombol, input, dan label.
   - **Compose Molecules**: Gabungkan atom menjadi kartu atau bar pencarian.
   - **Assemble Organisms**: Susun menjadi Navbar atau Footer.
   - **Deploy**: Build proyek menjadi file statis untuk di-upload ke server (Vercel/Netlify).

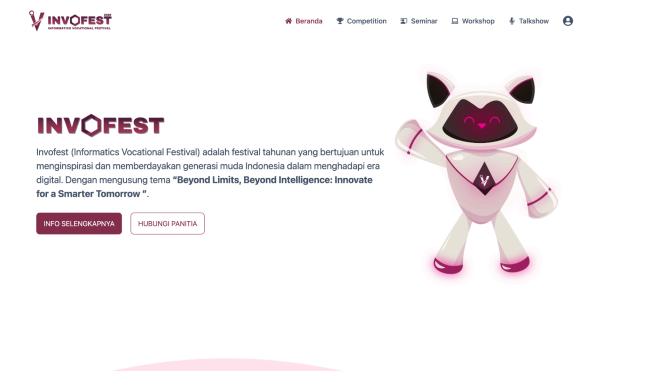
Case Study (Re-Engineering Invofest)

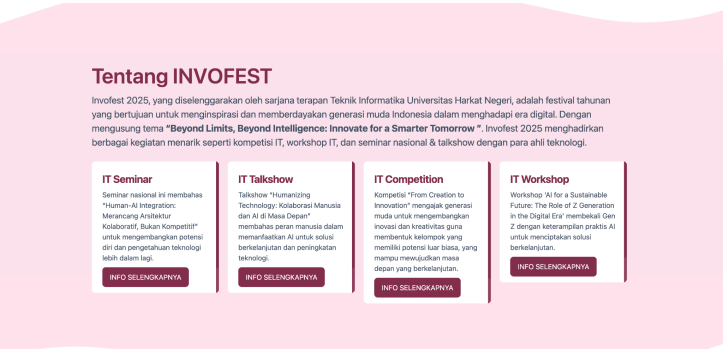
Pada bagian ini, kita akan melakukan "bedah desain" pada website **Invofest**. Tujuannya adalah melatih insting kalian dalam melihat sebuah website bukan sebagai satu halaman utuh, melainkan sebagai kumpulan komponen yang saling tersusun.

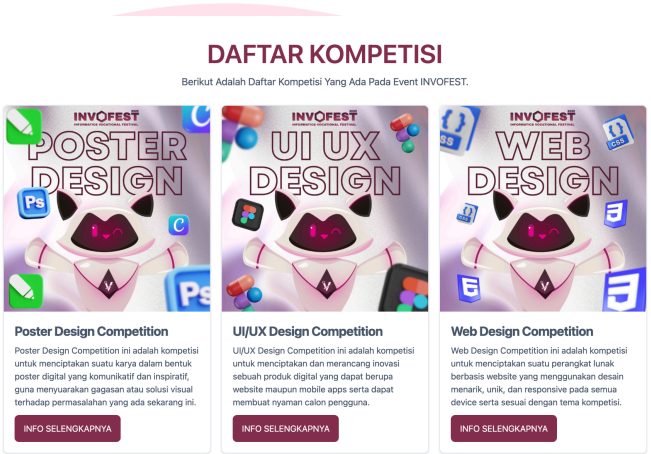
1. Strategi Component Breakdown

Sebelum mulai menulis kode, seorang *developer* profesional akan melakukan analisis visual. Mari kita bedah halaman Invofest menjadi tiga kategori:

- **Atoms (Komponen Terkecil):** Button, Ikon, Label, Input.
- **Molecules (Gabungan Atoms):** NavLink (Ikon + Teks), SearchBar, CardItem.
- **Organisms (Gabungan Molecules):** Navbar, Sidebar, Footer, Hero Section.







2. **Breakdown Komponen (Atomic Design)**
   Berdasarkan analisa UI pada website Invofest, kita membagi antarmuka menjadi beberapa bagian:

Nama Komponen	Kategori	Jenis Data (Props)	State
NavLink	Atom	`label`, `href`, **`icon`**, **`isActive`**	-
Button	Atom	label, variant, onClick	-
Heading	Atom	title, level (h1-h3)	-
Card Informasi	Molecule	icon, title, desc	-
Card Pembicara	Molecule	name, role, imgUrl	-
Collapse (FAQ)	Molecule	question, answer	isOpen (boolean)
Navbar / Header	Organism	menuItems[] (Array of Objects)	isMobileOpen (boolean)

3. **Analisa Detail Komponen Navigasi**
   a. **NavLink (Atom)**
      - **Active NavLink**

      - **Default NavLink**

Ini adalah unit terkecil dalam navigasi.

- **Peran:** Menangani perpindahan halaman atau *scroll* ke section tertentu (anchor link).
- **Kenapa Atom?** Karena ia berdiri sendiri dan hanya memiliki satu fungsi tunggal yaitu sebagai tautan teks.

Implementasi Code: NavLink dengan Icon & Active State

Kita akan menggunakan **Lucide React** (pustaka ikon standar industri yang ringan) untuk bagian ikon. Kita akan membedakan tampilan antara tautan yang sedang aktif dan yang tidak.

```
// components/ui/NavLink.tsx
interface NavLinkProps {
 label: string;
 href: string;
 icon?: React.ReactNode;
 isActive?: boolean;
}
export const NavLink: React.FC<NavLinkProps> = ({
 label,
 href,
 icon,
 isActive = false,
}) => {
 const activeStyle = "text-red-900";
 const defaultStyle = "text-slate-600 hover:text-red-900";
 return (
   <a
     href={href}
     className={`flex items-center gap-2 px-4 py-2 font-medium transition-all
duration-200 ${
       isActive ? activeStyle : defaultStyle
     }`}
   >
     {icon && <span className="w-5 h-5">{icon}</span>}
```

```
    <span>{label}</span>
  </a>
 );
};
```

b. Navbar / Header (Organism)

Ini adalah komponen tingkat tinggi yang menggabungkan beberapa atom dan molekul.

- **Komposisi:** Di dalamnya terdapat Logo (Atom), kumpulan NavLink (Atoms), dan Button daftar (Atom).
- **Penggunaan State:** Mengapa ada State isMobileOpen? Karena pada tampilan mobile (HP), menu navigasi biasanya disembunyikan di balik ikon "hamburger". State ini berfungsi untuk menyimpan kondisi apakah menu tersebut sedang **Tampil (True)** atau **Sembunyi (False)**.

Contoh Penggunaan di Header (Organism)

Di sini kita akan mensimulasikan bagaimana mengirimkan ikon dan menentukan mana link yang sedang aktif.

```
// components/Header.tsx
import { Home, Info, Users, HelpCircle } from "lucide-react";
import { NavLink } from "./ui/NavLink";

export const Header: React.FC = () => {
 const currentPath = "#";

 const menuItems = [
   { label: "Beranda", href: "#", icon: <Home size={18} /> },
   { label: "Tentang", href: "#about", icon: <Info size={18} /> },
   { label: "Narasumber", href: "#speakers", icon: <Users size={18} /> },
   { label: "FAQ", href: "#faq", icon: <HelpCircle size={18} /> },
 ];

 return (
   <header className=" bg-white shadow-sm px-6 py-2">
     <div className="max-w-7xl mx-auto flex justify-between items-center gap-4">
       <div className="logo">
         <img
           src="https://www.invofest-harkatnegeri.com/assets/nav-logo.png"
           alt="logo"
           className="h-16"
         />
       </div>
       <div className="nav flex gap-2">
         {menuItems.map((item) => (
```

```
            <NavLink
              key={item.label}
              label={item.label}
              href={item.href}
              icon={item.icon}
              // Logika: Jika href sama dengan path saat ini, maka Active
              isActive={item.href === currentPath}
            />
          ))}
        </div>
      </div>
    </header>
  );
};

export default Header;
```

Tambahkan pada file utama

buka file App.tsx

```
import Header from "./components/Header";

function App() {
  return (
    <>
      <Header />
    </>
  );
}

export default App;
```

c. Komponen Button (Type-Safe)

Gunakan Tailwind CSS untuk styling yang cepat dan konsisten.

```
interface ButtonProps {
 label: string;
 variant?: "primary" | "outline";
 className?: string;
 onClick?: () => void;
}

export const Button: React.FC<ButtonProps> = ({
 label,
 variant = "primary",
 className,
}) => {
 const baseStyle =
   "px-10 py-3 rounded font-medium transition-all duration-200";
 const variantStyle =
   variant === "primary"
     ? "bg-red-900 text-white hover:bg-red-800"
     : "border border-red-900 text-red-900 hover:bg-red-100";
 return (
   <button className={`${baseStyle} ${variantStyle} ${className}`}>
     {label}
   </button>
 );
};

export default Button;
```

d. Card Pembicara (Reusing Atoms)

Komponen ini akan dipanggil berkali-kali menggunakan fungsi .map() di halaman utama.

```
interface SpeakerCardProps {
 name: string;
 role: string;
 imageUrl: string;
}

export const SpeakerCard: React.FC<SpeakerCardProps> = ({
 name,
 role,
 imageUrl,
}) => {
 return (
   <div className="cursor-pointer flex flex-col items-center gap-4
group">
     <div className="relative h-1/3 z-1">
       <img
         src={imageUrl}
         alt={name}
         className="h-64 w-64 rounded-full border-10 border-red-900
mx-auto group-hover:scale-105 transition-transform duration-300
object-cover"
       />
     </div>
     <div className="cursor-pointer flex flex-col items-center h-2/3
border-4 border-red-900 w-full p-4 rounded-lg relative py-10 shadow-xl
shadow-black/20 group-hover:shadow-2xl group-hover:shadow-black/40
transition-all duration-300">
       //card overlay
       <div className="absolute bottom-0 px-4 py-6 rounded-lg w-full
h-full flex flex-col items-center gap-2 group-hover:bg-red-200
backdrop-blur-sm transition-all duration-300"></div>
       <div className="absolute left-0 right-0 bottom-0 items-center
justify-center p-4 flex flex-col gap-2">
         <h3 className="text-2xl text-red-900 font-semibold">{name}</h3>
         <p className="text-sm text-gray-600 text-center">{role}</p>
       </div>
```

```
      </div>
    </div>
  );
};

export default SpeakerCard;
```

e. **Collapse FAQ (Managing State)**

Interaktivitas "Buka-Tutup" pada bagian FAQ dikelola secara internal oleh **State**.

```
import React, { useState } from "react";
import { ChevronDown } from "lucide-react";

interface CollapseProps {
 title: string;
 description: string;
}

export const Collapse: React.FC<CollapseProps> = ({ title, description
}) => {
 const [isOpen, setIsOpen] = useState(false);

 return (
   <div className="border border-gray-200 rounded-lg">
     <button
       onClick={() => setIsOpen(!isOpen)}
       className="cursor-pointer w-full px-4 py-3 flex items-center
gap-4 bg-white hover:bg-gray-50 transition-colors"
     >
       <div className="p-2 bg-gray-100">
         <ChevronDown
           size={20}
           className={`text-gray-600 transition-transform ${
             isOpen ? "rotate-180" : ""
           }`}
         />
       </div>
       <span className="text-lg font-semibold
text-gray-800">{title}</span>
     </button>

     {isOpen && (
```

```
        <div className="px-4 py-3 bg-gray-50 border-t border-gray-200">
          <p className="text-gray-700">{description}</p>
        </div>
      )}
    </div>
  );
};
```

4. Perakitan Halaman Utama (Organism/Template)

Pada tahap ini, `App.tsx` berfungsi sebagai *Layout Manager*. Ia memanggil `Navbar`, mendistribusikan data ke `SpeakerCard`, dan mengelola status buka-tutup pada `Collapse`.

```
import Header from "./components/Header";
import Button from "./components/ui/Button";
import { Collapse } from "./components/ui/Collapse";
import SpeakerCard from "./components/ui/SpeakerCard";
import { Card } from "./components/ui/Card";

function App() {
 const speakers = [
   {
     name: "Dery Agung Triyadi",
     role: "Aws Indonesia",
     imageUrl:
       "https://www.invofest-harkatnegeri.com/assets/seminar/Seminar%20Dery.png",
   },
   {
     name: "Sowam Habibi",
     role: "Google Indonesia",
     imageUrl:
       "https://www.invofest-harkatnegeri.com/assets/seminar/seminar%20sowam.png",
   },
   {
     name: "Lhuqita Fazry",
     role: "Mobile Development Developer, Founder Rumah Coding Indonesia",
     imageUrl:
       "https://www.invofest-harkatnegeri.com/assets/workshop/workshop%20mobile.png",
   },
 ];

 const collapseItems = [
   {
     title: "Apa itu Invofest?",
     description:
       "Invofest (Informatics Vocational Festival) adalah festival tahunan yang
bertujuan untuk menginspirasi dan memberdayakan generasi muda Indonesia dalam menghadapi
era digital. Dengan mengusung tema "Beyond Limits, Beyond Intelligence: Innovate for a
Smarter Tomorrow ".",
```

```
    },
    {
      title: "Kapan dan di mana Invofest akan diselenggarakan?",
      description:
        "Invofest akan diselenggarakan pada tanggal 15-17 November 2024 di Jakarta
Convention Center (JCC), Jakarta, Indonesia.",
    },
    {
      title: "Siapa saja yang dapat mengikuti Invofest?",
      description:
        "Invofest terbuka untuk semua kalangan, terutama mahasiswa, pelajar, profesional
muda, dan siapa saja yang tertarik dengan teknologi dan inovasi. Acara ini dirancang
untuk memberikan inspirasi dan pengetahuan kepada semua peserta, tanpa memandang latar
belakang atau tingkat keahlian.",
    },
  ];

  const cardItems = [
    {
      title: "IT Seminar",
      description:
        "Seminar nasional ini membahas "Human-AI Integration: Merancang Arsitektur
Kolaboratif, Bukan Kompetitif" untuk mengembangkan potensi diri dan pengetahuan
teknologi lebih dalam lagi.",
    },
    {
      title: "IT Talkshow",
      description:
        "Talkshow "Humanizing Technology: Kolaborasi Manusia dan AI di Masa Depan"
membahas peran manusia dalam memanfaatkan AI untuk solusi berkelanjutan dan peningkatan
teknologi.",
    },
    {
      title: "IT Competition",
      description:
        "Kompetisi "From Creation to Innovation" mengajak generasi muda untuk
mengembangkan inovasi dan kreativitas guna membentuk kelompok yang memiliki potensi luar
biasa, yang mampu mewujudkan masa depan yang berkelanjutan.",
    },
    {
      title: "IT Workshop",
      description:
        "Workshop 'AI for a Sustainable Future: The Role of Z Generation in the Digital
Era' membekali Gen Z dengan keterampilan praktis AI untuk menciptakan solusi
berkelanjutan.",
    },
  ];

  return (
    <>
      <Header />

      <div className="max-w-7xl mx-auto">
```

```
<section
  id="hero"
  className="py-10 flex gap-10 justify-between items-center "
>
  <div className="w-2/3 flex flex-col gap-6">
    <img
      src="https://www.invofest-harkatnegeri.com/assets/text-image.png"
      alt=""
      className="w-96"
    />
    <p>
      Invofest (Informatics Vocational Festival) adalah festival tahunan
      yang bertujuan untuk menginspirasi dan memberdayakan generasi muda
      Indonesia dalam menghadapi era digital. Dengan mengusung tema
      "Beyond Limits, Beyond Intelligence: Innovate for a Smarter
      Tomorrow ".
    </p>

    <div className="flex gap-3">
      <Button label="Info Selengkapnya" variant="primary" />
      <Button label="Hubungi Panitia" variant="outline" />
    </div>
  </div>
  <div className="w-1/3">
    <img
      src="https://www.invofest-harkatnegeri.com/assets/Maskot-Hero.png"
      alt=""
    />
  </div>
</section>

<section id="speaker" className="py-24">
  <div className="grid grid-cols-1 md:grid-cols-3 gap-10 px-3">
    {speakers.map((speaker, index) => (
      <SpeakerCard
        key={index}
        name={speaker.name}
        role={speaker.role}
        imageUrl={speaker.imageUrl}
      />
    ))}
  </div>
</section>

<section
  id="cards"
  className="py-24 grid grid-cols-1 md:grid-cols-2 gap-10 px-3"
>
  {cardItems.map((item, index) => (
    <Card key={index} className="w-full">
      <h3 className="text-2xl text-red-900 mb-4">{item.title}</h3>
      <p>{item.description}</p>
      <Button
```

```
              label="Info Selengkapnya"
              variant="primary"
              className="mt-4"
            />
          </Card>
        ))}
      </section>

      <section id="collapse" className="py-24 flex flex-col gap-2 px-3">
        {collapseItems.map((item, index) => (
          <Collapse
            key={index}
            title={item.title}
            description={item.description}
          />
        ))}
      </section>
    </div>
  </>
 );
}

export default App;
```

Konsep Dasar Deploy React

Project React (Vite) itu:

BUKAN di-upload source code mentah
Tapi harus di-**build dulu jadi static file**

Hasil Build itu apa?

Setelah build, project berubah jadi:

```
HTML + CSS + JS (static)
```

Inilah yang di-upload ke server

1. Build Project

Jalankan di local:

```
npm run build
```

Hasilnya

Akan muncul folder:

dist/

Isi contoh:

```
dist/
 ├── index.html
 ├── assets/
 │    ├── index-xxxxx.js
 │    ├── index-xxxxx.css
```

2. File yang Harus Diupload

HANYA upload isi folder dist

JANGAN upload:

- node_modules
- src
- package.json

3. Cara Deploy ke Server

Ada beberapa metode. Saya jelaskan dari yang paling umum:

- **Shared Hosting / cPanel**

  Langkah:

  1. Build project
  2. Buka **File Manager (cPanel)**
  3. Masuk ke:

     public_html/

  4. Upload isi folder dist

- **Deploy Paling Mudah (Rekomendasi)**

  Kalau untuk belajar / portfolio:

  - ☐ **Vercel**

    Langkah:

    1. Push ke GitHub
    2. Login ke Vercel
    3. Import repo
    4. Deploy

    Auto detect Vite

- ☐ **Netlify**
  - Drag & drop folder dist
  - Selesai dalam 1 menit

- ☐ **Cloudflare Pages**
  - Cepat & gratis
  - Cocok production

4. Preview Build

Memastikan hasil production (dist) berjalan normal sebelum upload ke server.

- **Cara 1 (PALING DIREKOMENDASIKAN): vite preview**

  Setelah build:

  ```
  npm run build
  npm run preview
  ```

  Hasilnya

  Terminal akan menampilkan:

  Local: http://localhost:4173/

  Buka di browser → ini adalah **simulasi production**

5. Perbedaan Penting

Mode	Perintah	Kegunaan
Dev	npm run dev	untuk coding
Preview	npm run preview	untuk cek hasil build
Build	npm run build	generate dist